

Hardware Requirements

This section outlines the core development boards and sensors used across the projects in this book, highlighting why each was selected and what it contributes to the learning experience. We've prioritized accessible, well-supported platforms that are widely available, but we also include more specialized boards where necessary to demonstrate real-world capabilities like computer vision, sound generation, or sensor fusion. Think of this as your starter kit for building intelligent devices at the edge.

In addition to the embedded hardware, you'll need a laptop or desktop computer capable of running Python scripts and training machine learning models. Any modern system, whether running Windows, macOS, or Linux, should be sufficient, as long as it supports Python 3.x and can handle basic ML training workloads.

While this book focuses on embedded AI, the reality is that much of the model training, especially for deep learning, is performed on laptops, on desktops, or in the cloud. These environments offer the processing power and memory required to handle large datasets and complex computations that would be impractical on microcontrollers. Once trained, models are then quantized, compressed, and optimized for deployment on embedded hardware. This separation of concerns (train on the desktop, deploy at the edge) is a pragmatic and widely adopted workflow.

Each project has a parts list so you can get exactly what you need when you need it. You do not have to buy everything in advance, and you do not need most of the boards to get started.

A good portion of the book needs no hardware at all. Around a quarter of the chapters can be followed entirely on a laptop, using tools like TensorFlow and scikit-learn. Beyond that, two-thirds of the chapters can be completed with just two boards: the Arduino UNO and the Raspberry Pi Pico. Together, these cost around \$35, and the Pico on its own is under \$5. Add the Nano 33 BLE Sense, and you can build close to three-quarters of the projects in the book for a combined cost of around \$70.

These three boards (the UNO, the Pico, and the Nano 33 BLE Sense) are reused across many projects, so they are the ones worth investing in up front. The remaining boards are each used in a single chapter, so you can buy them only if you decide to build those particular projects.

If you want to plan ahead, the full set of boards and platforms used across all projects is:

- Arduino UNO (R3 or R4)
- Arduino Nano 33 BLE Sense (R1 or R2)
- Arduino Nicla Vision
- Arduino Nicla Voice
- Raspberry Pi Pico (RP2040)

- Raspberry Pi Pico 2 (RP2350)
- STM32F401VE motherboard (MKI109V3) with ST ISM330BX 6-axis IMU (MKI245KA)

Browse the list, decide which projects you want to build first, and assemble the gear as you go. Some projects use Reefwing custom PCBs that need to be fabricated and assembled before use. The Gerber files are included in the book's project repository, and you can upload them directly to your preferred PCB manufacturer. Allow one to two weeks for fabrication and shipping, depending on your location and supplier.

Software Requirements

A diverse set of software tools is needed to complete the projects in this book. These tools span the full stack of embedded AI development—from compiling C++ firmware in the Arduino IDE, to training neural networks in Python using VS Code, to deploying models with TensorFlow Lite for Microcontrollers, and capturing sensor data using terminal utilities. We've aimed to select open source, widely available tools, and where possible, platform-agnostic options compatible with Windows, macOS, and Linux. While we recommend tools like Visual Studio Code, Terminal, and specific Python libraries, feel free to substitute your preferred equivalents. Download links for the referenced software are available in the resources. The following list summarizes the primary software requirements:

Languages and Runtimes

- Python
- C/C++

IDEs and Editors

- Arduino IDE
- VS Code (with Pico SDK extension for Chapters 10 and 11)
- OpenMV IDE
- MEMS-Studio

Toolchains and Build Tools

- AVR-GCC and AVRDUDE
- Make
- Arduino CLI
- Pico SDK

Frameworks

- TensorFlow Lite for Microcontrollers
- Edge Impulse

- RNNoise
- CMSIS-DSP

Utilities

- CoolTerm
- Audacity
- FluidSynth
- TensorBoard
- Netron
- xxd
- Syntiant Uploader

Windows Arm-based systems, at the time of writing, have issues using driver-dependent or emulated tools, which impacts the use of Arduino IDE (DFU drivers), AVR-GCC, AVRDUDE, FluidSynth, Audacity, MEMS-Studio, CoolTerm, and OpenMV IDE.

Using Python on a Windows Arm PC

If you're running Windows on an Arm64 machine, note that Python's native Arm64 builds do not yet support all required libraries. At the time of writing, pandas was not available for Python 3.11, 3.12, or 3.13 on native Windows Arm64, and Python 3.10 has no native Arm64 build at all.

The workaround is to install the standard x86 version of Python, which runs under emulation on Arm64 hardware. While training will be slower than on a native x86 machine, all libraries will install and function correctly.

Tools for Assembly and Debugging

In addition to electronic components and microcontrollers, you'll need a few basic tools to build and troubleshoot the projects in this book. Nothing fancy—just the kind of gear you'd find in a typical maker's toolbox. A good set of screwdrivers, long-nosed pliers, wire cutters, and jumper wires will go a long way. If you plan to assemble your own PCBs, a soldering iron and some soldering accessories will also be necessary.

We will use the following debugging tools:

Multimeter Quickly verifies voltages, currents, continuity, and resistance/capacitance.

LCR meter Measures inductance, capacitance, and resistance, providing accurate component-level values. Some of these meters can also characterize transistors and diodes.

Logic analyzer Debugs digital communication interfaces (SPI, I2C, UART). The problem with logic analyzers is that they clean up the

signal and make it look like you are getting crisp changes in voltage, even if the actual signals on your hardware are noisy, slow, or distorted. As a result, subtle issues like signal noise, ringing, overshoot, slow rise/fall times, or marginal voltage levels can easily go unnoticed. This is why you need an oscilloscope.

Oscilloscope Examines signals in real time, debugs communication protocols, and observes transient phenomena. Shows the true analog waveform, revealing problems such as noise, glitches, signal degradation, slow edges, or voltage-level issues.

Hardware JTAG/SWD and in-circuit debuggers Allows stepping through code, inspecting registers and memory, and setting breakpoints.

None of these tools are essential, but they do make life easier when things go wrong. If you can only afford one, get a multimeter. For the signal generator project, an oscilloscope is very useful for observing waveform shape, frequency, and timing. A multimeter supports basic checks, but an oscilloscope provides the visibility needed to properly validate signal generation.